02_연구계획서

연구 계획서 — Research Proposal

전문화된 다중 SLM 협업 시스템에서 최적 통신 프로토콜과 집단 실수 메모리를 활용한 자기 개선 프레임워크 연구 A Self-Improving Framework via Optimal Communication Protocols and Collective Error Memory in Specialized Multi-SLM Collaboration

항목	내용
지원 분야	인공지능 / 자연어처리 / 멀티에이전트 시스템
핵심 키워드	SLM 협업, 멀티에이전트, 통신 프로토콜, 집단 메모리, 자기 개선
지원 학위	석사 과정 (박사 진학 희망)
작성일	2026년 4월

---

1. 연구 배경 및 동기

1.1 선행 시스템에서 식별한 두 가지 한계

본 연구는 두 개의 선행 시스템 구현 경험에서 도출되었다.

선행 시스템 1 — OLaLA (가짜뉴스 판별 다중 에이전트) LangGraph 9-stage 파이프라인 기반으로 증거 수집(Stage 1–5) → 검증(Stage 6–8) → Judge(Stage 9) 구조. 핵심은 Stage 6–7 의 **이중 대립 에이전트(Dual-Adversarial Agents)** — 두 SLM 이 같은 증거를 각각 지지/반박 관점으로 독립 분석한 뒤 Judge 가 종합하는 구조다. 결과: 단일 LLM 대비 hallucination rate 약 20% 감소 + 89% 정확도.

선행 시스템 2 — CompaniAI (소프트웨어 자동 생성) 사용자 자연어 요청 → CTO 에이전트 전략 수립 → 작업 분해 → 역할별 SLM 병렬 실행. SharedWorkspace + MessageQueue + Emergency Meeting + Peer Review 의 4개 협업 메커니즘 구현. 25개 모듈 + 470+ 단위 테스트, Mac Mini M4 16GB 환경에서 동작.

두 시스템 운영 과정에서 다음 두 한계가 식별되었다:

한계 1 — 단일 LLM 의존 구조의 비효율 ChatDev, MetaGPT 등 기존 멀티에이전트 연구는 형식상 여러 에이전트를 두지만 실제로는 GPT-4 같은 단일 대형 모델을 system prompt 만 바꿔 반복 호출하는 구조다. 이는 (a) API 비용, (b) 재현성 부족 (개인 연구자/소형 lab), (c) 단일 모델 한계 (한 번의 잘못된 reasoning 이 모든 에이전트를 오염) 의 세 문제를 가진다. OLaLA 의 결과가 시사하는 것은 SLM 도 역할 분리가 명확하면 협업 가능하다는 점인데, 학계의 기본 가정은 이 가능성을 충분히 검증하지 않은 상태다.

한계 2 — 정적 에이전트 (경험 누적 부재) 두 시스템 모두에서 동일 패턴의 hallucination / schema 위반이 task 마다 반복 관측되었다. ExpeL [Zhao+2024] 이 단일 에이전트 경험 누적은 다뤘으나, **팀 단위 집단 메모리** 는 미연구 영역이다. 한 에이전트의 실수를 팀 전체가 공유하는 메모리 메커니즘이 제안되지 않았다.

이 두 한계의 교차점이 본 연구의 출발점이다.

1.2 기존 연구의 한계

관련 논문을 읽으면서 정리한 현황입니다.

- **단일 LLM 의존 구조**: ChatDev, MetaGPT, AutoGPT 등 기존 멀티에이전트 시스템은 비용이 높고 개인 연구자가 재현하기 어렵습니다. 또한 단일 모델 한계 (한 번의 잘못된 reasoning 이 모든 에이전트를 오염) 가 극복되지 않습니다.

- **정적인 에이전트**: 실행 중 발생한 실수가 누적되지 않습니다. 같은 task 를 다시 풀면 같은 실수를 반복합니다. ExpeL [Zhao+2024] 이 단일 에이전트 경험 누적을 다뤘지만 다중 에이전트 팀의 집단 메모리는 거의 다뤄지지 않았습니다.
- **통신 프로토콜의 미시 비교 부재**: ProtocolBench [arXiv:2510.17149] 가 최근 비교 시도를 했지만, 통신 프로토콜과 메모리를 함께 변수로 다루는 연구는 아직 부족합니다.

본 연구는 이 두 문제(다중 SLM 협업 가능성 + 집단 메모리)를 **통신 프로토콜이라는 공통 axis 위에서** 함께 다루는 것이 차별점입니다.

---

2. 연구 목표 및 연구 질문

2.1 연구 목표

목표	내용
O1	동일 컴퓨팅 비용에서 역할 특화 SLM 협업이 단일 LLM 대비 우위를 가지는 task 유형의 식별
O2	정확성과 속도를 동시에 만족하는 에이전트 간 통신 프로토콜 설계 (Protocol D 신규 제안)
O3	팀 전체가 공유하는 집단 실수 메모리 (Collective Error Memory, CEM) 메커니즘 설계 + 반복 오류 감소 검증

2.2 핵심 연구 질문 (RQ)

RQ1 — 다중 SLM vs 단일 LLM 의 효율성 비교

동일 컴퓨팅 비용(FLOPs) 조건에서, 역할 특화 다중 SLM 팀은 단일 LLM 대비 어떤 task 유형에서 우위를 보이는가?

가설: Task 가 **(a) 명확히 분해 가능**하고 **(b) 역할 전문성이 중요**할수록, 다중 SLM 팀이 같은 FLOPs 에서 단일 LLM 대비 정확도/속도 우위를 보일 것이다. 단순 단일-step 추론은 단일 LLM 이 우위.

RQ2 — 최적 통신 프로토콜의 설계 원칙

에이전트 간 통신에서 정확성과 속도를 동시에 만족하는 프로토콜의 설계 원칙은 무엇인가?

프로토콜	방식	가설
Protocol A	자연어 전체 전송	자연스럽지만 토큰 비효율
Protocol B	JSON 구조화	토큰 절감, 자유도 손실 (현재 `compani_ai` 가 사용)
Protocol C	CoT 요약 공유	근거 투명성, 토큰 증가
Protocol D ★	**중요도 기반 자동 선택**	신규 제안 — 효과 검증 필요

Protocol D 의 중요도 점수 (초기 가설):

```
importance_score(message) = 0.4 × relevance + 0.3 × recency + 0.3 × severity
```

- relevance: cosine similarity with current task embedding
- recency: exponential decay (낮은 task_index → 낮은 가중치)
- severity: 1.0 (CRITICAL) / 0.7 (MAJOR) / 0.3 (MINOR) / 0.1 (INFO)

가중치는 초기값이며, 그리드 서치 또는 메타러닝으로 최적화 예정.

RQ3 — 집단 실수 메모리 (CEM) 의 효과

팀 전체가 공유하는 episode 단위 실수 메모리는 반복 오류율을 얼마나 감소시키는가? 자동 규칙 추출은 효과가 있는가?

가설: - (H3-1) CEM 사용 시 동일 task suite 의 반복 오류율이 N회 실행 후 단조 감소 - (H3-2) 같은 실수가 임계값(N=3) 이상 반복되면 자동 추출된 방지 규칙이 추가적인 오류 감소를 가져옴 - (H3-3) 개인 메모리(per-agent)보다 공유 메모리(shared CEM)가 cross-agent 오류 감소에 효과적

---

3. 시스템 아키텍처 (현재 구현 + 확장 계획)

3.1 4계층 구조

기존 `compani_ai_dev3` 의 구현 위에 통신/메모리 계층을 명시화하는 형태로 확장합니다.

```
┌──────────────────────────────────────────────────────────────┐
│  L1. 오케스트레이션 (Orchestration)                            │
│  - CTO Agent: 전략 수립 → 작업 분해 → 진행 모니터링 → 메모리 조회│
│  - 모델: qwen3:8b (현재) → Llama 3.1 70B (확장)                 │
└──────────────────────────────────────────────────────────────┘
                                  │
                                  ↓
┌──────────────────────────────────────────────────────────────┐
│  L2. 실행 (Specialized SLM Execution)                          │
│  - Backend: gemma4:e4b (코드 생성)                              │
│  - Frontend: gemma4:e4b (UI 생성)                               │
│  - MLOps: llama3.2:3b (Docker / CI 생성)                        │
│  - 병렬: Python asyncio + Per-role concurrency limiter          │
└──────────────────────────────────────────────────────────────┘
                                  │
                                  ↓
┌──────────────────────────────────────────────────────────────┐
│  L3. 통신 (Communication Protocols) ★ RQ2                       │
│  - SharedWorkspace: WorkItem 상태 broadcast (EventBus)          │
│  - MessageQueue: Q&A point-to-point                             │
│  - Emergency Meeting: blocking 충돌 시 합의 회의                 │
│  - Peer Review: 코드 리뷰 + severity 분류                        │
│  - Protocol A/B/C/D 비교 실험 인터페이스 (예정)                   │
└──────────────────────────────────────────────────────────────┘
                                  │
                                  ↓
┌──────────────────────────────────────────────────────────────┐
│  L4. 집단 실수 메모리 (Collective Error Memory) ★ RQ3           │
│  - Storage: Qdrant (벡터) + SQLite (메타데이터)                  │
│  - 자동 캡처: ErrorCode + peer review severity                   │
│  - 검색: 새 task 시작 시 유사 과거 실수 retrieve → prompt 주입    │
│  - 규칙 승격: 동일 패턴 N회 반복 시 system prompt 고정 삽입       │
│  - 현재: 미구현 (석사 1학기 핵심 작업)                            │
└──────────────────────────────────────────────────────────────┘
```

3.2 현재 구현 수준 (`compani_ai_dev3` 기반)

구성요소	상태	비고
L1 오케스트레이션	작동	CTO 의 strategy + decompose, Stage Gate Meeting, qwen3:8b
L2 SLM 실행	작동	

구성요소	상태	비고
		Backend/Frontend/MLOps, asyncio 병렬, per-role concurrency limiter
L3 통신 (현 형태)	작동	SharedWorkspace + MessageQueue + Emergency Meeting + Peer Review
L3 Protocol A/B/C/D 비교	설계 중	현재 B(JSON) 만 구현. A/C/D 는 abstraction layer 추가 필요
L4 CEM	미구현	석사 1학기 1순위

전체 코드: GitHub `AlBe-lang/compani_ai`, 약 25개 어댑터/애플리케이션 모듈, 470+ 단위 테스트, Mac Mini M4 16GB 에서 동작 검증.

3.3 통신 프로토콜 비교 실험 설계 (RQ2)

프로토콜	메시지 인코딩	라우팅	토큰 추정 (per task)
Protocol A	자유 자연어	broadcast	~2000 토큰
Protocol B	JSON schema	targeted	~800 토큰
Protocol C	CoT + summary	broadcast	~1400 토큰
Protocol D	importance-filtered (top-K)	scored routing	~600 토큰 (예상)

각 프로토콜은 동일한 SLM 팀 + 동일한 task suite 에서 비교. 측정: task 완성률, 통신 토큰 수, end-to-end latency, error propagation rate.

3.4 집단 실수 메모리 (CEM) 설계 (RQ3)

```
ErrorEvent {
  error_code: str           # E-LLM-TIMEOUT / E-PARSE-JSON / ...
  agent_id: str
  task_id: str
  context: str (embedded)   # 발생 시점 컨텍스트
  resolution: str | None    # 사람/자동 수정 결과
  severity: MINOR | MAJOR | CRITICAL
  recurrence_count: int     # 같은 패턴 누적 횟수
  ts: datetime
}
```

검색 흐름: 1. 새 task 시작 시, task description 임베딩 → Qdrant 유사도 검색 (top-5) 2. 검색된 실수 이벤트 → CTO 의 system prompt 에 "과거 유사 task 에서 발생한 실수" 섹션으로 주입 3. SLM 실행 시 해당 컨텍스트도 함께 전달

규칙 승격 (Rule Promotion): - 같은 `error_code` + 유사한 컨텍스트 (cosine sim > 0.85) 가 **N=3** 회 반복되면 - 자동으로 `prevention_rule` 로 변환 — 예: "Pydantic v2 에서는 `from_orm` 대신 `model_validate` 를 사용할 것" - 해당 규칙은 향후 모든 SLM 의 system prompt 에 고정 삽입

---

4. 실험 설계

4.1 실험 환경

항목	사양
메인 환경	Mac Mini M4 16GB (현재)
확장 환경	연구실 GPU 자원 활용 희망 (RTX 6000 / A100 등)
로컬 모델	qwen3:8b / gemma4:e4b / llama3.2:3b (Ollama)
비교 모델	Llama 3.1 70B / GPT-4 (외부 API 또는 연구실 자원)
인프라	Python 3.11 + FastAPI + Pydantic v2 + asyncio + SQLite WAL + Qdrant
코드 저장소	github.com/AlBe-lang/compani_ai

4.2 실험 1 — 단일 LLM vs 다중 SLM (RQ1, 3-4 개월차)

변수	값
Baseline	Llama 3.1 70B 단일 호출
Treatment	qwen3:8b (CTO) + gemma4:e4b × 2 + llama3.2:3b (총 ~17B 파라미터)
Task suite	(1) compani_ai 도메인: Todo / Blog / Guestbook / 회의록 시스템 / 간단 Wiki — 5개 reference projects (2) HumanEval (코드 생성 표준 벤치) (3) OLaLA fact-check task 일부
측정	정확도/완성도, end-to-end latency, FLOPs, 토큰 비용
제어	동일한 시스템 프롬프트, 동일한 random seed (where applicable)

예상 결과 시나리오: - 단순 단일-step task (예: 단일 함수 생성) → LLM 우위 - 다중 모듈 task (예: backend + frontend + deploy 통합) → SLM 팀 우위 (가설)

4.3 실험 2 — 통신 프로토콜 비교 (RQ2, 5–7 개월차)

변수	값
고정	SLM 팀 구성 (실험 1의 다중 SLM 그대로)
변수	Protocol A / B / C / D
Task suite	실험 1과 동일
측정	(a) 통신 토큰 수 — 메시지 총합 (b) 오류 전파율 — 한 에이전트 실수가 다음 에이전트로 전파된 비율 (c) Task 완성률 (d) End-to-end latency
추가 분석	

변수	값
	Protocol D 의 importance_score 가중치 그리드 서치 ($3^3 \times 3 = 27$개 조합)

4.4 실험 3 — 집단 실수 메모리 효과 검증 (RQ3, 8–10 개월차)

변수	값
고정	통신 프로토콜 = 실험 2 의 최적 (예: Protocol D)
변수	(a) 메모리 없음 (b) per-agent 메모리 (c) shared CEM (d) shared CEM + 자동 규칙 승격
Task suite	동일 task 를 N=10 회 반복 실행
측정	반복 오류율 감소 곡선, 수렴 횟수, false positive rate (잘못 학습된 규칙)

핵심 가설 검증: - (H3-1) (a) → (c) 로 갈수록 N회 후 오류율 단조 감소 - (H3-2) (c) vs (d) 비교에서 (d) 가 추가 감소 - (H3-3) (b) vs (c) 비교에서 cross-agent 오류는 (c) 에서 더 큰 감소

4.5 검증 도메인의 일반화

`compani_ai` 의 코드 생성 도메인 외에: - **OLaLA** — 가짜뉴스 판별 task 로 같은 SLM 팀 구조의 일반화 검증 - **표준 벤치** — HumanEval, ProtocolBench 의 task suite 일부 채택

이 두 추가 도메인을 통해 결과의 generalizability 를 부분 확보.

---

5. 관련 연구 및 이론적 위치

5.1 핵심 참조 문헌

논문	본 연구와의 관계
Qian et al. (2024). **ChatDev**. ACL 2024	가장 비슷한 구조 (소프트웨어 회사 시뮬). 단, 단일 LLM 의존 → 본 연구는 SLM 팀으로 대체
Wang et al. (2024). **Survey on SLMs + LLMs Collaboration**. arXiv: 2510.13890	SLM 협업 이론적 framing 의 출발점
ProtocolBench. arXiv: 2510.17149	통신 프로토콜 비교 방법론 — 본 연구의 RQ2 실험 설계 기반
Zhao et al. (2024). **ExpeL: Experiential Learners**. AAAI 2024	단일 에이전트 경험 누적 — 본 연구의 RQ3 출발점, **팀 단위 확장** 이 차별
MemRL: Self-Evolving Agents. arXiv:2601.03192	자기 개선 메모리 최근 동향
Episodic Memory Missing Piece. arXiv:2502.06975	Episode 단위 메모리의 학술적 motivation

논문	본 연구와의 관계
Hong et al. (2023). **MetaGPT**. ICLR 2024	SOP 기반 멀티에이전트 — 본 연구의 동적 task decomposition 과 대비

5.2 본 연구의 학술적 위치

기존 연구 대부분은 (1) 단일 LLM 의 다중 호출 또는 (2) 단일 에이전트의 메모리/경험 학습 중 한 axis 만 다룹니다. 본 연구는 다음 세 axis 를 동시에 변수로 다루는 것이 차별 contribution 입니다:

1. **모델 차원**: 단일 LLM vs 역할 특화 다중 SLM
2. **통신 차원**: 자연어 / 구조화 / CoT / importance-filtered
3. **메모리 차원**: 없음 / per-agent / shared CEM / CEM + auto-rule

세 axis 의 상호작용 자체가 미연구 영역이며, 특히 **메모리 품질이 최적 통신 프로토콜에 의해 결정된다는 가설** 은 새로운 학술 contribution 이 될 수 있다고 봅니다.

---

6. 연구 일정 (12개월)

기간	단계	핵심 작업	산출물
0–2개월	준비	핵심 논문 정독 / 실험 환경 정비 / `compani_ai` 코드 정리 (R-06A pre-commit, R-11A HOT_RELOADABLE 분류 등 미완 항목)	정리된 baseline 구현, 논문 노트
3–4개월	실험 1	단일 LLM vs SLM 팀 비교 (RQ1)	첫 실험 결과 + 워크샵 논문 초안 시도
5–7개월	실험 2	Protocol A/B/C/D 비교 + Protocol D 구현 (RQ2)	통신 프로토콜 비교 분석
8–10개월	실험 3	CEM 시스템 구현 + 반복 오류 감소 검증 (RQ3)	메모리 시스템 + 효과 분석
11–12개월	통합	결과 종합 / 논문 초안 작성	학회 투고 시도 (KCC, ACL/EMNLP/AAAI workshop)

---

7. 본 연구의 학술적 / 실무적 기여

7.1 학술적 기여

- 다중 SLM 팀의 우위 task 유형 식별 → SLM 협업 이론의 경험적 근거 추가
- 통신 프로토콜과 집단 메모리의 상호작용을 처음으로 통합 분석
- Protocol D (importance-filtered) 의 신규 제안 + 검증

7.2 실무적 기여

- 외부 유료 API 없이 16GB 로컬 환경에서 재현 가능한 멀티에이전트 시스템 — **개인 연구자 / 오픈소스 커뮤니티 / 자원 제한적 산업 환경** 에 직접적 가치
- `compani_ai` 오픈소스 — 학회 발표 후 재현 코드 공개 예정 (GitHub `AlBe-lang/compani_ai`)

8. 연구 한계 및 보완 계획

본 연구의 설계상 한계와 보완 방향:

1. **실험 자원 한계** — 16GB Mac 환경 단독으로는 GPT-4 baseline 비교나 대규모 task suite 가 어렵다. 연구실 자원 또는 클라우드 학생 크레딧 (GCP, AWS) 활용이 필요하다.
2. **검증 도메인의 generalizability** — `compani_ai` (코드 생성) + OLaLA (가짜뉴스) 두 도메인만으로는 결과의 일반화 주장에 한계가 있다. HumanEval / ProtocolBench 의 task suite 를 추가 검증 채널로 사용한다.
3. **CEM 의 noise 문제** — false positive 로 학습된 방지 규칙이 시스템 품질을 떨어뜨릴 가능성이 있다. confidence threshold + 외부 검증 (코드 생성 task 의 테스트 통과율 등) 을 안전장치로 설계하지만, 실 데이터 누적 후 튜닝이 필요하다.
4. **Protocol D 가중치의 임의성** — 초기값 (0.4 / 0.3 / 0.3) 은 직관 기반이다. 그리드 서치 ($3^3 \times 3 = 27$ 조합) + 메타러닝으로 가중치를 최적화하는 별도 ablation 이 RQ2 의 sub-question 으로 포함된다.

참고문헌

[1] Qian, C. et al. (2024). ChatDev: Communicative Agents for Software Development. ACL 2024. [2] Hong, S. et al. (2024). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. ICLR 2024. [3] Wang, F. et al. (2024). A Survey on Collaborating SLMs and LLMs. arXiv:2510.13890. [4] ProtocolBench: Which LLM Multi-Agent Protocol Is Best? arXiv:2510.17149. [5] Zhao, A. et al. (2024). ExpeL: LLM Agents Are Experiential Learners. AAAI 2024. [6] MemRL: Self-Evolving Agents via Runtime RL. arXiv:2601.03192. [7] Episodic Memory is the Missing Piece for LLM Agents. arXiv:2502.06975.

2026년 4월